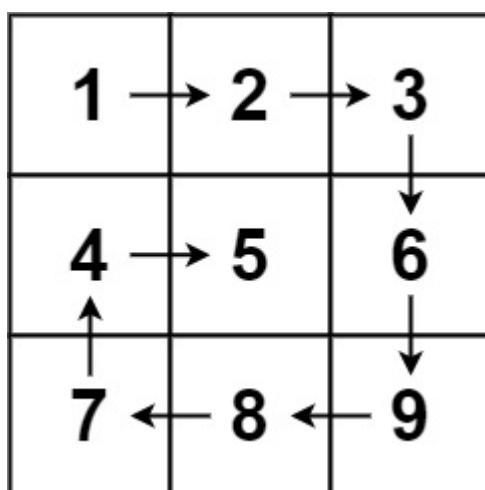


54. 螺旋矩阵

给你一个 m 行 n 列的矩阵 `matrix`，请按照 **顺时针螺旋顺序**，返回矩阵中的所有元素。

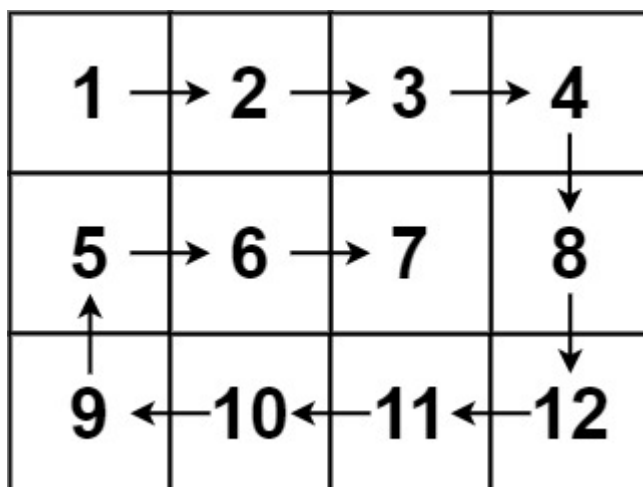
示例 1:



输入: `matrix = [[1,2,3],[4,5,6],[7,8,9]]`

输出: `[1,2,3,6,9,8,7,4,5]`

示例 2:



输入: matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]

输出: [1,2,3,4,8,12,11,10,9,5,6,7]

2. 解决方法

好的，我们来解析“螺旋矩阵”这个问题 (LeetCode 54)。

1. 问题理解

- **目标：** 给定一个 $m \times n$ 的矩阵 `matrix`，要求按照**顺时针螺旋**的顺序返回矩阵中的所有元素。
- **输入：** 一个二维整数数组 `matrix`。
- **输出：** 一个包含矩阵所有元素的一维列表 `List<Integer>`，元素的顺序是螺旋遍历的顺序。
- **螺旋顺序：** 从左上角开始，先向右遍历一行，然后向下遍历一列，然后向左遍历一行，然后向上遍历一列，接着缩小边界，重复这个过程，直到遍历完所有元素。

2. 普通人的思路（模拟螺旋路径）

最直观的想法就是模拟人在矩阵中按照螺旋路径行走的过程。我们需要记录当前的位置和方向，并适时地改变方向。

• 思路：

1. 初始化一个结果列表 `result`。
2. 定义矩阵的边界: `top`, `bottom`, `left`, `right`。初始时, `top = 0`, `bottom = m-1`, `left = 0`, `right = n-1`。
3. 使用一个循环，只要还有元素未被访问（可以通过比较 `left <= right` 和 `top <= bottom` 来判断）。
4. 在循环内部，按顺序执行四个步骤（模拟一圈）：
 - **向右走：** 从 `left` 到 `right` 遍历第 `top` 行，将元素加入 `result`。然后 `top` 边界向下移动一位 (`top++`)。
 - **向下走：** 从 `top` 到 `bottom` 遍历第 `right` 列，将元素加入 `result`。然后 `right` 边界向左移动一位 (`right--`)。

- **向左走：** 在向左和向上走之前，需要检查边界是否有效（`top <= bottom` 和 `left <= right`），因为可能在走完向右或向下后就已经遍历完了所有元素（例如，只有一行或一列的情况）。如果有效，从 `right` 到 `left` 遍历第 `bottom` 行，将元素加入 `result`。然后 `bottom` 边界向上移动一位（`bottom--`）。
- **向上走：** 同样检查边界是否有效。如果有效，从 `bottom` 到 `top` 遍历第 `left` 列，将元素加入 `result`。然后 `left` 边界向右移动一位（`left++`）。

5. 循环结束后，`result` 列表就包含了所有按螺旋顺序排列的元素。

- **优点：** 直接模拟了过程，比较好理解。
- **缺点：** 需要仔细处理边界条件和循环终止条件，特别是单行/单列或奇数行列的情况。

3. 更好的方法（边界收缩法 - O(1) 额外空间，除了结果列表）

这个方法本质上就是对上述模拟思路的结构化实现，通过维护四个边界变量来控制遍历范围。

- **核心思想：** 定义四个变量 `top`，`bottom`，`left`，`right` 表示当前需要遍历的矩阵部分的边界。每一轮循环处理一圈螺旋，然后收缩边界。
- **步骤：**
 1. 初始化结果列表 `result`。
 2. 获取矩阵维度 `m` 和 `n`。处理空矩阵或无效矩阵的边界情况。
 3. 初始化边界：`top = 0`，`bottom = m - 1`，`left = 0`，`right = n - 1`。
 4. 进入主循环，条件是 `top <= bottom` 且 `left <= right`。
 - **从左到右（上边界）：** 遍历 `matrix[top][j]` 从 `j = left` 到 `right`，加入 `result`。然后 `top++`。
 - **从上到下（右边界）：** 遍历 `matrix[i][right]` 从 `i = top` 到 `bottom`，加入 `result`。然后 `right--`。
 - **检查边界并从右到左（下边界）：** 如果 `top <= bottom`（防止单行情况重复遍历），遍历 `matrix[bottom][j]` 从 `j = right` 到 `left`，加入 `result`。然后 `bottom--`。
 - **检查边界并从下到上（左边界）：** 如果 `left <= right`（防止单列情况重复遍历），遍历 `matrix[i][left]` 从 `i = bottom` 到 `top`，加入 `result`。然后 `left++`。
 5. 循环结束，返回 `result`。
- **复杂度：**
 - 时间复杂度： $O(m * n)$ 。每个元素被访问一次。

- 。空间复杂度： $O(1)$ 。除了存储结果的列表外，只使用了常数个额外变量。

4. 示例演示（边界收缩法）

`matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]` ($m=3, n=3$)

1. 初始化: `result = []`, `top = 0`, `bottom = 2`, `left = 0`, `right = 2`。

2. 第一圈:

- 向右: `j` 从 0 到 2 遍历 `matrix[0][j]` → `result = [1, 2, 3]` . `top = 1` .
- 向下: `i` 从 1 到 2 遍历 `matrix[i][2]` → `result = [1, 2, 3, 6, 9]` . `right = 1` .
- 向左 (检查 `top <= bottom` → `1 <= 2`): `j` 从 1 到 0 遍历 `matrix[2][j]` → `result = [1, 2, 3, 6, 9, 8, 7]` . `bottom = 1` .
- 向上 (检查 `left <= right` → `0 <= 1`): `i` 从 1 到 1 遍历 `matrix[i][0]` → `result = [1, 2, 3, 6, 9, 8, 7, 4]` . `left = 1` .

3. 第二圈 (检查 `top <= bottom` → `1 <= 1`, `left <= right` → `1 <= 1`):

- 向右: `j` 从 1 到 1 遍历 `matrix[1][j]` → `result = [1, 2, 3, 6, 9, 8, 7, 4, 5]` . `top = 2` .
- 向下 (检查 `top <= bottom` → `2 <= 1`, 不满足, 跳过)
- 向左 (检查 `top <= bottom` → `2 <= 1`, 不满足, 跳过)
- 向上 (检查 `left <= right` → `1 <= 1`, 但 `top > bottom`, 循环条件 `i = bottom` 到 `top` 无法执行, 跳过)

4. 循环结束条件: `top = 2`, `bottom = 1` → `top > bottom`。循环终止。

5. 返回: `[1, 2, 3, 6, 9, 8, 7, 4, 5]`。

5. 代码实现（边界收缩法）

```
import java.util.ArrayList;
import java.util.List;

/*
 * @lc app=leetcode.cn id=54 lang=java
 * @lcpr version=30204
 *
 * [54] 螺旋矩阵
 */
```

```

// @lcpr-template-start

// @lcpr-template-end
// @lc code=start
class Solution {
    public List<Integer> spiralOrder(int[][] matrix) {
        List<Integer> result = new ArrayList<>();
        if (matrix == null || matrix.length == 0 || matrix[0].length == 0) {
            return result;
        }

        int m = matrix.length;
        int n = matrix[0].length;
        int top = 0, bottom = m - 1, left = 0, right = n - 1;

        while (top <= bottom && left <= right) {
            // 从左到右 (上边界)
            for (int j = left; j <= right; j++) {
                result.add(matrix[top][j]);
            }
            top++; // 上边界下移

            // 从上到下 (右边界)
            for (int i = top; i <= bottom; i++) {
                result.add(matrix[i][right]);
            }
            right--; // 右边界左移

            // 检查边界，防止单行或单列时重复添加
            if (top <= bottom) {
                // 从右到左 (下边界)
                for (int j = right; j >= left; j--) {
                    result.add(matrix[bottom][j]);
                }
                bottom--; // 下边界上移
            }

            if (left <= right) {

```

```

        // 从下到上 (左边界)
        for (int i = bottom; i >= top; i--) {
            result.add(matrix[i][left]);
        }
        left++; // 左边界右移
    }
}

return result;
}
}
// @lc code=end

```

```

/*
// @lcpr case=start
// [[1,2,3],[4,5,6],[7,8,9]]\n
// @lcpr case=end

// @lcpr case=start
// [[1,2,3,4],[5,6,7,8],[9,10,11,12]]\n
// @lcpr case=end

*/

```

6. 如何在面试中讲解

1. **确认问题：** "好的，我们需要按顺时针螺旋顺序遍历一个 $m \times n$ 矩阵，并返回包含所有元素的列表。"
2. **核心思路 - 边界控制：** "这个问题的常用方法是模拟螺旋的路径，并通过维护四个边界变量 `top` , `bottom` , `left` , `right` 来控制每一圈遍历的范围。"
3. **分步讲解算法：** "我们可以用一个 `while` 循环，只要上下边界和左右边界没有交错 (`top <= bottom` 且 `left <= right`)，就执行一圈螺旋遍历：
 - 首先，从 `left` 到 `right` 遍历上边界 (第 `top` 行)，然后将 `top` 加 1。
 - 接着，从 `top` 到 `bottom` 遍历右边界 (第 `right` 列)，然后将 `right` 减 1。

- 然后，**需要检查一下** `top` 是否仍然小于等于 `bottom`，如果满足，再从 `right` 到 `left` 遍历下边界（第 `bottom` 行），然后将 `bottom` 减 1。这一步检查是为了处理只有一行的情况。
 - 最后，**再检查一下** `left` 是否仍然小于等于 `right`，如果满足，再从 `bottom` 到 `top` 遍历左边界（第 `left` 列），然后将 `left` 加 1。这一步检查是为了处理只有一列的情况。"
4. **循环终止：** "当 `top > bottom` 或 `left > right` 时，表示所有元素都已遍历完毕，循环结束。"
 5. **复杂度：** "这个方法每个元素只访问一次，所以时间复杂度是 $O(m * n)$ 。除了结果列表外，只使用了常数个额外变量，空间复杂度是 $O(1)$ 。"
 6. **示例（可选）：** "可以用一个 3x3 的矩阵来演示边界如何收缩以及遍历的顺序..."

这样讲解清晰地展示了算法的逻辑、边界处理和效率。

找到具有 2 个许可证类型的类似代码